

MODULE 04

SYLLABUS: Sounds: Making Sound Using tone() and Speaker, Making Sound Sequences. Interrupts: Hardware Interrupts, Trade-offs Between Polling and Interrupting, Arduino Hardware Interrupt Capabilities, Hardware Debounced Button Interrupt Circuit, Hardware interrupt example code, Timer Interrupts, Timer interrupt example.

TO WATCH VIDEO CLASSES
OF THIS COURSE: [TAP HERE OR](#)
[SCAN QR CODE](#)

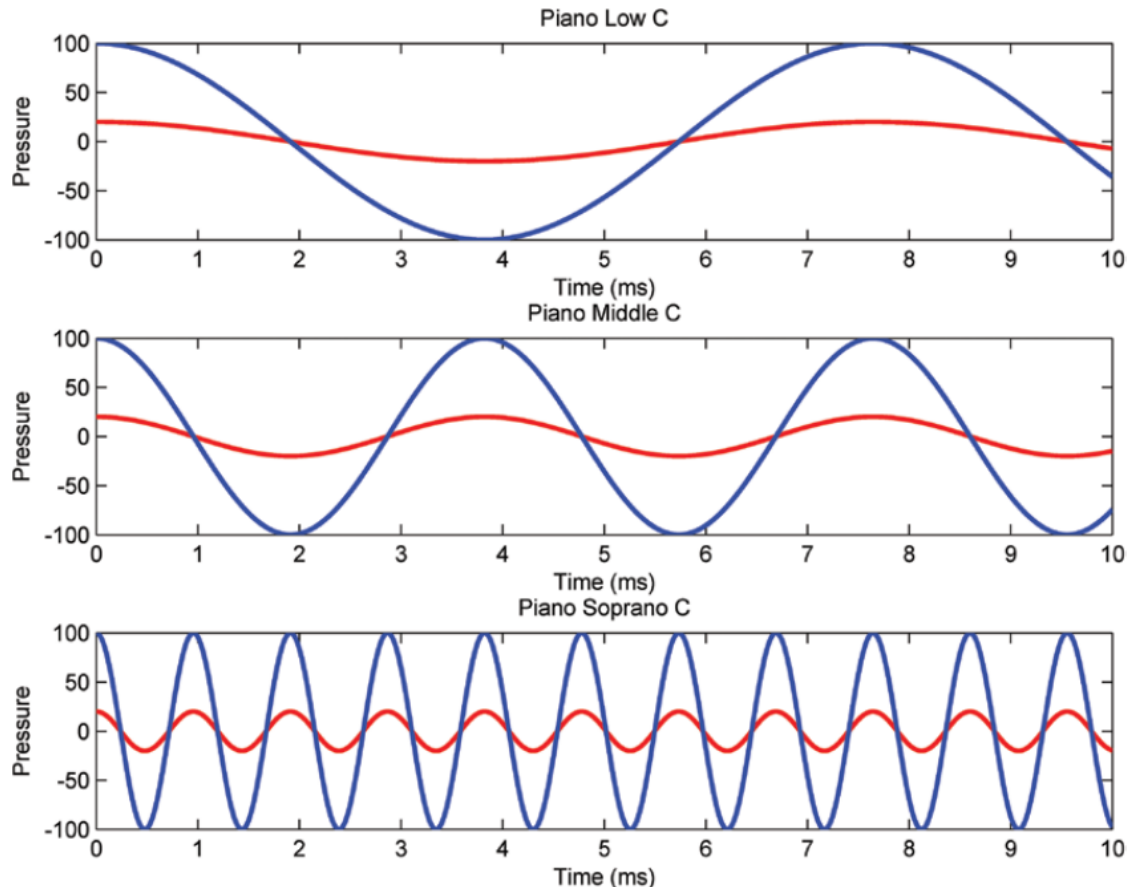


SOUNDS AND MUSIC

Sound is transmitted through air as a **pressure wave**. A vibrating object (speaker, bell, string) causes air particles to vibrate, which transfer that energy to neighboring particles, eventually reaching the human eardrum and being perceived as sound.

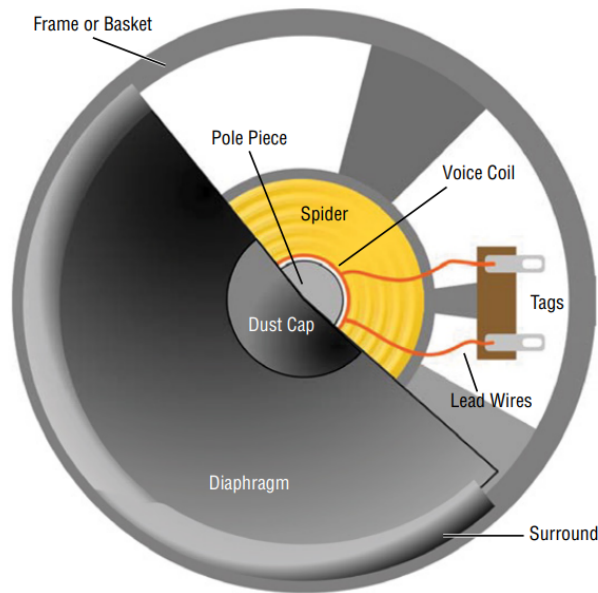
Two properties of sound are controllable from an Arduino:

Property	Physical Effect	Musical Effect
Frequency	How fast air particles vibrate (cycles per second, Hz)	Pitch — higher frequency = higher pitch
Amplitude	Magnitude of vibration	Loudness — higher amplitude = louder sound



Each complete oscillation in the graph represents one cycle. The width of one cycle = $1/\text{frequency}$. For middle C: period = $1/261.63 = 3.822$ ms.

How a Speaker Produces Sound



A speaker consists of:

- A **permanent magnet** mounted at the rear
- A **voice coil** wound around the pole piece, attached to the diaphragm

When an alternating electrical signal (square wave from Arduino) is fed to the voice coil, it creates a **changing magnetic field**. This field is alternately attracted to and repelled by the permanent magnet, causing the voice coil and diaphragm to vibrate back and forth. This vibration compresses and rarefies the air in front of it, creating a pressure wave — sound.

Important: The Arduino (except the Due with its DAC) generates only **square waves**, not sine waves. Square waves still create audible sound, but are not as smooth or natural-sounding as a true sine wave.

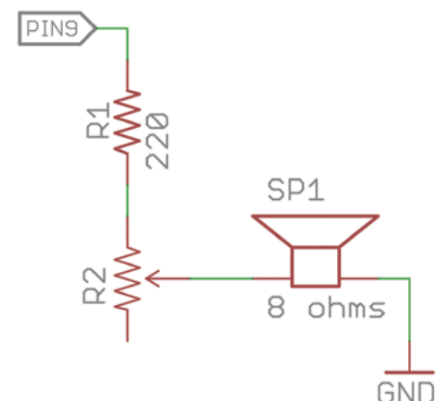
Wiring a Speaker to Arduino

Current Limiting Resistor — Why It Is Needed

Each Arduino I/O pin can supply a maximum of **40 mA**. A standard 8Ω speaker must have a series resistor to prevent exceeding this limit.

Resistor calculation using Ohm's Law:

- Supply voltage: 5V
- Maximum current: $40\text{ mA} = 0.04\text{ A}$
- Minimum total resistance: $R = V/I = 5/0.04 = 125\ \Omega$
- Speaker's internal resistance: $8\ \Omega$
- Minimum external resistor: $125 - 8 = 117\ \Omega$
- Nearest standard value (higher): **$150\ \Omega$ or $220\ \Omega$** (either works)



A **potentiometer in series** with the fixed resistor allows dynamic volume control. Using only the wiper and one end terminal (not all three), the full potentiometer resistance adds to the series when turned toward maximum resistance (quietest), and zero resistance adds when turned toward minimum (loudest).

Speakers have **no polarity** — they can be connected in either direction.

The tone() Function

The `tone()` function generates a **square wave** of a specified frequency on any Arduino output pin. It uses one of the ATmega's hardware timers internally, so the tone plays in the **background** while the Arduino continues executing other code.

Syntax

```
tone(pin, frequency);           // Plays continuously until noTone()
tone(pin, frequency, duration); // Plays for duration milliseconds, then stops
noTone(pin);                     // Stops tone on specified pin
```

Argument	Type	Description
pin	int	Output pin connected to speaker
frequency	unsigned int	Frequency in Hz
duration	unsigned long	Optional — duration in milliseconds

Important Limitations of tone()

Limitation	Detail
Only one pin at a time	<code>tone()</code> can generate sound on only one pin simultaneously
Timer conflict	Uses the same hardware timer as PWM on pins 3 and 11 — <code>analogWrite()</code> on those pins does not work correctly while <code>tone()</code> is active
Square wave only	Produces square waves, not sine waves — suitable for tones, not music playback
No WAV playback	For audio file playback, an external music shield or the Arduino Due's DAC is needed

Using a Pitch Definition File

Musical notes are defined by their frequencies. The Arduino IDE provides a header file (`pitches.h`) that maps human-readable note names to their frequencies.

Including the Header File

```
#include "pitches.h" // Add this line at top of sketch
```

Place `pitches.h` in the same folder as your `.ino` sketch file. The IDE automatically shows it as a second tab.

Sample definitions from pitches.h:

```
#define NOTE_C4 262 // Middle C, 262 Hz
#define NOTE_D4 294
#define NOTE_E4 330
#define NOTE_A4 440
// ...and so on for all notes across octaves
```

The `#define` preprocessor directive creates a named constant — the compiler replaces `NOTE_C4` with 262 wherever it appears.

Alternatively, individual notes can be defined directly in the sketch without the header file:

```
#define NOTE_C 262
#define NOTE_D 294
#define NOTE_E 330
#define NOTE_G 392
#define NOTE_A 440
```

Arrays — Storing Sequences of Data

An **array** is a collection of variables of the same type stored sequentially in memory under a single name. Arrays are essential for storing note sequences and durations.

Array Declaration and Initialization

```
// Declare empty array of 4 integers
int numbers[4];

// Declare and initialize — both forms are equivalent
int numbers[4] = {-7, 0, 6, 234};
int numbers[] = {-7, 0, 6, 234}; // Length inferred from values
```

Accessing Array Elements

Arrays are **zero-indexed** — the first element is at index 0.

```
analogWrite(9, numbers[2]); // Access third element (index 2)
numbers[2] = 10;           // Modify third element
```

Using sizeof() for Array Length

```
int len = sizeof(notes) / sizeof(notes[0]); // Total bytes / bytes per element
```

This avoids hardcoding the array length in the for loop.

Making Sound Sequences — Music Player

Note and Duration Arrays

Two parallel arrays store the music:

- notes[] — pitch of each note (0 = rest/silence)
- times[] — duration of each note in milliseconds

Both arrays must be the **same length**.

```
// Quarter notes = 250ms, Eighth notes = 125ms
// 0 in notes array = musical rest (silence)
```

```
int notes[] = {
  NOTE_A4, NOTE_E3, NOTE_A4, 0,
  NOTE_A4, NOTE_E3, NOTE_A4, 0,
  NOTE_E4, NOTE_D4, NOTE_C4, NOTE_B4, NOTE_A4, NOTE_B4, NOTE_C4, NOTE_D4,
  NOTE_E4, NOTE_E3, NOTE_A4, 0
};
```

```
int times[] = {
  250, 250, 250, 250,
  250, 250, 250, 250,
  125, 125, 125, 125, 125, 125, 125, 125,
  250, 250, 250, 250
};
```

Complete Music Player Program

```
#include "pitches.h"    // Frequency definitions for all notes

const int SPEAKER = 9;  // Speaker with 220Ω resistor on pin 9

int notes[] = {
  NOTE_A4, NOTE_E3, NOTE_A4, 0,
  NOTE_A4, NOTE_E3, NOTE_A4, 0,
  NOTE_E4, NOTE_D4, NOTE_C4, NOTE_B4, NOTE_A4, NOTE_B4, NOTE_C4, NOTE_D4,
  NOTE_E4, NOTE_E3, NOTE_A4, 0
};

int times[] = {
  250, 250, 250, 250,
  250, 250, 250, 250,
  125, 125, 125, 125, 125, 125, 125, 125,
  250, 250, 250, 250
};

void setup() {
  // Play each note once at startup
  for (int i = 0; i < 20; i++) {
    tone(SPEAKER, notes[i], times[i]); // Play note for its duration
    delay(times[i]);                  // Wait for note to finish
    // tone() plays in background; delay() holds until note completes
    // Without delay(), next note would start before current one ends
  }
  // Press Reset button to replay
}

void loop() {
  // Empty — song plays once in setup()
}
```

Why `delay(times[i])` is needed after `tone()`: `tone()` starts the sound and immediately returns — it runs in the background using a hardware timer. Without the matching `delay()`, the loop would advance to the next note before the current one finishes, causing all notes to play simultaneously or get cut short.

Rest handling: When `notes[i] = 0`, `tone(SPEAKER, 0, times[i])` produces no sound — the delay still runs, creating the silence of a musical rest.

Pentatonic Micro Piano

The Pentatonic Scale

The **pentatonic scale** consists of **five notes per octave** (C, D, E, G, A). Notes in this scale have **minimal dissonance** — any combination of them sounds harmonious. This makes it ideal for a simple piano where multiple notes might be played in any order.

Hardware

Five buttons with 10k Ω pull-down resistors connected to digital pins 3–7. Speaker on pin 9. No debouncing needed — the note plays only while the button is held, so bouncing simply creates brief repeated tone calls at the same frequency.

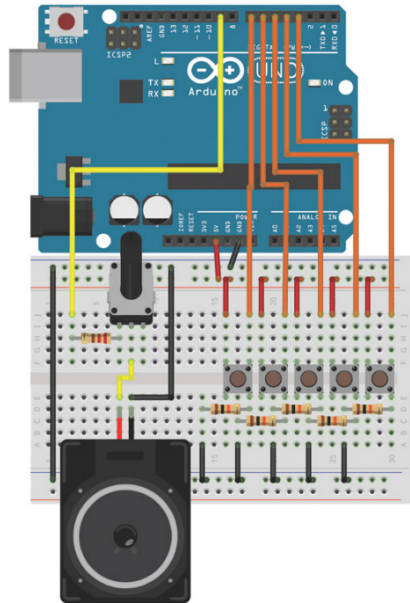


Fig: Five buttons each with one terminal to 5V and the other terminal through a 10k Ω pull-down resistor to GND and to Arduino pins 7,6,5,4,3. Speaker connected through 220 Ω resistor from pin 9 to GND.)

Micro Piano Program

```
// Pentatonic Piano — C D E G A
#define NOTE_C 262 // Hz
#define NOTE_D 294
#define NOTE_E 330
#define NOTE_G 392
#define NOTE_A 440

const int SPEAKER = 9; // Speaker on pin 9
const int BUTTON_C = 7; // Button for C
const int BUTTON_D = 6;
const int BUTTON_E = 5;
const int BUTTON_G = 4;
const int BUTTON_A = 3;

void setup() {
  // tone() sets pin as output automatically — no pinMode() needed
}

void loop() {
  // Each while() plays the tone as long as the button is held
  while (digitalRead(BUTTON_C)) tone(SPEAKER, NOTE_C); // No duration = continuous
  while (digitalRead(BUTTON_D)) tone(SPEAKER, NOTE_D);
  while (digitalRead(BUTTON_E)) tone(SPEAKER, NOTE_E);
  while (digitalRead(BUTTON_G)) tone(SPEAKER, NOTE_G);
  while (digitalRead(BUTTON_A)) tone(SPEAKER, NOTE_A);

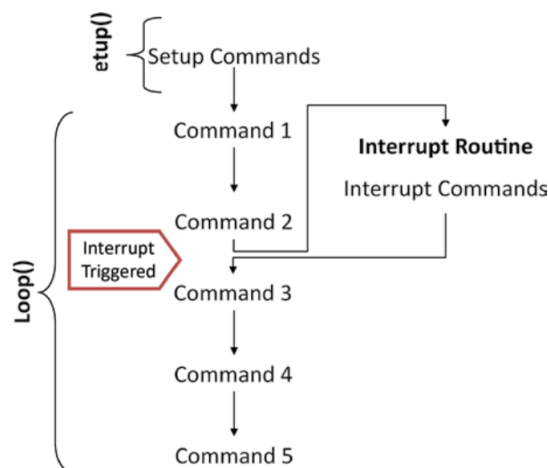
  noTone(SPEAKER); // Stop sound when all buttons are released
}
```

Key programming points:

Concept	Explanation
tone() without duration	Plays continuously until noTone() is called
while(digitalRead(pin))	Loops as long as button is HIGH (pressed); digitalRead() returns Boolean directly
Single-line loop body	Curly braces {} are optional when loop/if body is exactly one statement
noTone(SPEAKER)	Stops any active tone — called after all button checks when no button is pressed

INTERRUPTS

All Arduino programs written so far have been **synchronous** — the processor executes instructions one after another in a fixed sequence. This creates a fundamental problem: when the processor is busy executing a slow operation (such as a delay() inside a for loop), it cannot simultaneously check for external inputs like button presses. Events that occur during this busy period are simply **missed**.



An **interrupt** solves this problem. An interrupt is a hardware mechanism that allows the processor to **stop its current execution**, run a special short function called an **Interrupt Service Routine (ISR)**, and then **resume exactly where it left off** — all automatically, regardless of where the main program was when the event occurred.

Trade-offs Between Polling and Interrupting

Both polling and interrupts detect external input events. They are not universally better or worse than each other — the right choice depends on the application.

Polling

Polling means repeatedly checking the state of an input pin inside loop() using digitalRead().

```
void loop() {
  if (digitalRead(BUTTON) == HIGH) {
    // Handle button press
  }
  // Other code here
}
```

The button is only checked **once per loop iteration**. If other code in the loop takes a long time (e.g., a fade with delay()), button presses during that time are missed.

Interrupts

An interrupt-driven input is checked **asynchronously by hardware** — the processor is notified immediately when the event occurs, regardless of what it is currently doing.

Comparison Table

Feature	Polling	Hardware Interrupt
Implementation ease	Simpler — just <code>digitalRead()</code>	Slightly more code — <code>attachInterrupt()</code>
Hardware requirement	Same as interrupt for most inputs	RC + Schmitt trigger needed for bouncy buttons
Responsiveness	Misses events during slow code	Always responds immediately
Bouncy inputs	Can use software debounce	Must use hardware debounce (no <code>delay()</code> in ISR)
Multitasking	Not possible with slow loops	Enables pseudo-multitasking
Fast signal capture	May miss short pulses	Essential — rotary encoders, fast pulses
CPU efficiency	Wastes cycles checking continuously	Efficient — responds only when event occurs
Use case	Simple, fast loops	Slow loops, time-critical events

Key rule: If your loop executes quickly and missing the occasional input is acceptable — use polling. If your loop has slow operations (`delay()`, motor fades) or if missing any input is unacceptable (rotary encoder pulses) — use interrupts.

Arduino Hardware Interrupt Capabilities

Interrupt-Capable Pins on the Uno

Not all Arduino pins can function as hardware interrupts. On the **Arduino Uno** (ATmega328P):

Interrupt Pin	Interrupt ID
Pin 2	Interrupt 0
Pin 3	Interrupt 1

Always use `digitalPinToInterrupt(pin)` instead of hardcoding the interrupt ID — this ensures code portability across different Arduino boards.

Trigger Modes

An interrupt can be configured to fire on different signal conditions:

Trigger Mode	When ISR Fires
LOW	Continuously while pin is LOW (fires repeatedly — use with caution)
CHANGE	Any time pin state changes (LOW→HIGH or HIGH→LOW)
RISING	When pin transitions from LOW to HIGH
FALLING	When pin transitions from HIGH to LOW

RISING and FALLING are the most commonly used — they fire exactly **once** per state transition, making them ideal for button presses.

attachInterrupt() — Syntax

```
attachInterrupt(digitalPinToInterrupt(pin), ISR_function, trigger_mode);
```

Argument	Description
digitalPinToInterrupt(pin)	Converts physical pin number to interrupt ID — use always
ISR_function	Name of the function to call when interrupt fires (no parentheses)
trigger_mode	LOW, CHANGE, RISING, or FALLING

Example:

```
attachInterrupt(digitalPinToInterrupt(2), toggleLED, RISING);  
// Calls toggleLED() every time pin 2 goes from LOW to HIGH
```

Volatile Variables

Any variable that is **modified inside an ISR** must be declared as volatile. This tells the compiler that this variable can change unexpectedly (outside normal program flow) and must not be cached or optimized away.

```
volatile int selectedLED = 9; // Can be changed by ISR at any time
```

Without volatile, the compiler may optimize the variable into a CPU register, making the main loop unaware of changes made by the ISR.

Rules for ISR Functions

ISR functions have strict constraints:

Rule	Reason
Must be short and fast	ISR blocks the main loop while executing
No delay() inside ISR	delay() relies on timer interrupts which are disabled during ISR
No Serial.print() inside ISR	Serial uses interrupts internally
No return value, no parameters	ISR signature: void functionName()
Only modify volatile variables	Ensures main loop sees the updated values

Hardware Debouncing for Interrupt Inputs

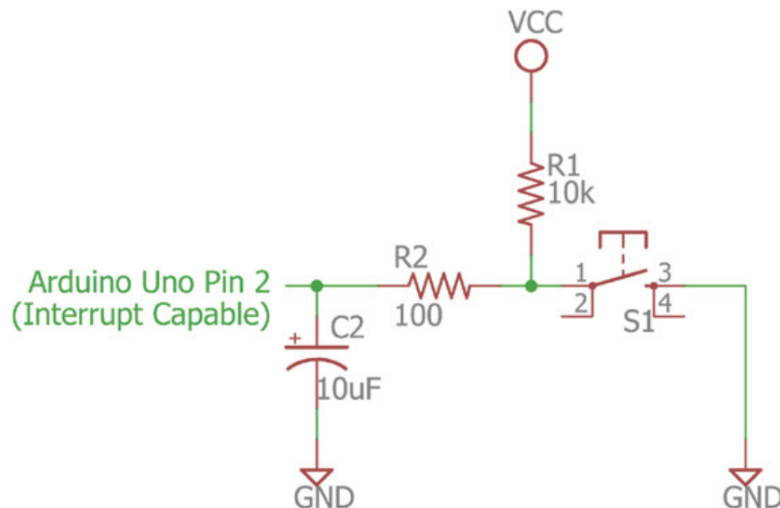
Why Software Debounce Fails with Interrupts

In Module 1, **software debouncing** used delay(5) to wait for a button to settle. Since delay() cannot be used inside an ISR, software debouncing is impossible with hardware interrupts. A bouncing button will trigger the ISR **multiple times** per physical press.

The solution is **hardware debouncing** using an RC filter and a Schmitt trigger.

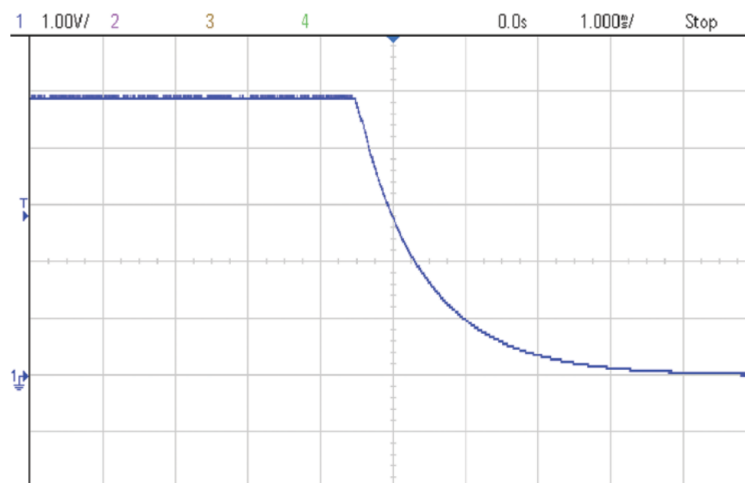
Step 1 — RC Filter Network

An **RC (Resistor-Capacitor) filter** is placed between the button and the Arduino input pin.



How it works:

- When button is **not pressed**: capacitor charges up to 5V through R1 and R2. Node B sits at logic HIGH.
- When button is **pressed**: capacitor begins discharging through R2 and the closed switch to GND. Because capacitors discharge gradually (not instantly), the signal at node B **slowly falls** rather than bouncing.
- The capacitor acts as an **energy reservoir** — it holds the output near 5V during the brief bounce openings of the switch, because those short gaps do not give enough time for the capacitor to fully recharge. Only after the switch has fully settled in the closed position does the capacitor completely discharge to 0V.

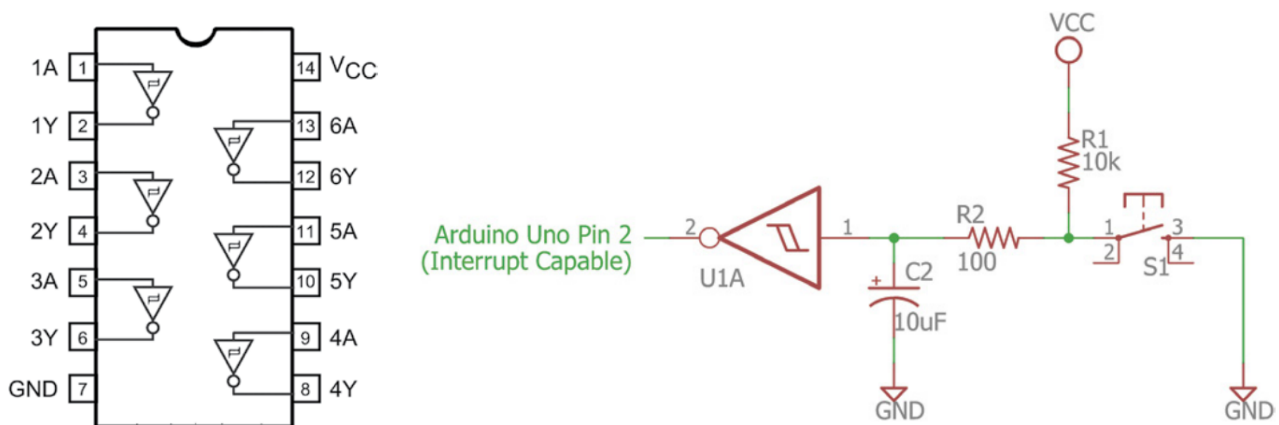


Step 2 — Inverting Schmitt Trigger (74AHCT14)

The RC filter creates a **slowly changing** voltage — not a sharp digital edge. Although the ATmega can handle this due to its built-in hysteresis (input must drop below $0.2 \times VCC$ for LOW, rise above $0.7 \times VCC$ for HIGH), a **Schmitt trigger** is added to produce a clean, sharp digital edge.

The **74AHCT14** is a hex **inverting Schmitt trigger** IC (6 independent inverters, one used here). It:

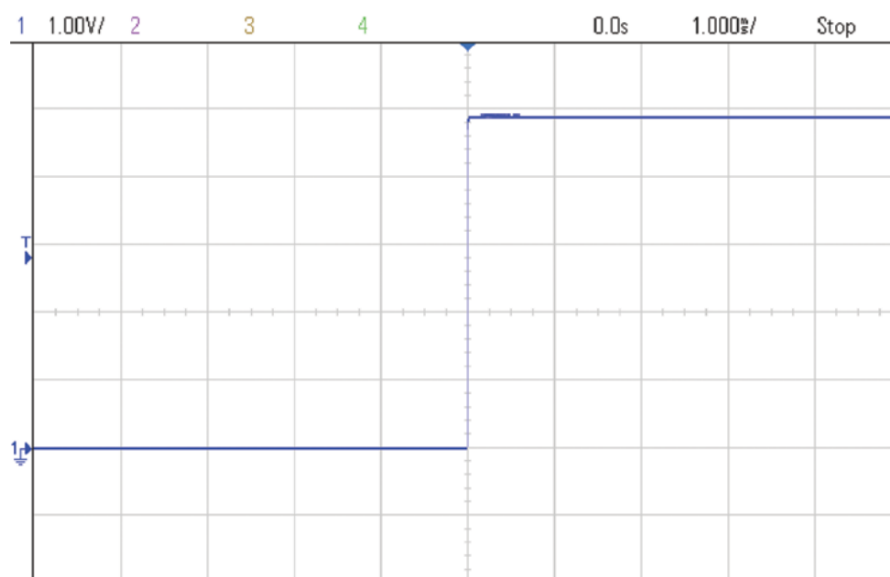
- Accepts the slowly-changing RC output
- Produces a sharp, clean digital edge at its output
- **Inverts** the signal (button pressed → output HIGH; released → output LOW)



Because the Schmitt trigger **inverts** the signal:

- Button not pressed → output is HIGH
- Button pressed → output goes LOW then back to HIGH when released
- The **rising edge** (LOW to HIGH) at the trigger output corresponds to the button being **released** — OR if using pull-down configuration, rising edge = button pressed

In the course example: the debounced output goes **HIGH when button is pressed** (after inversion). The interrupt is set to trigger on RISING edge.



Complete Debounce Circuit Summary

Button → Pull-up Resistor (R1)

→ Series Resistor (R2) → RC Filter Node → Capacitor (C) to GND

→ Schmitt Trigger Input → Schmitt Trigger Output → Arduino Pin 2

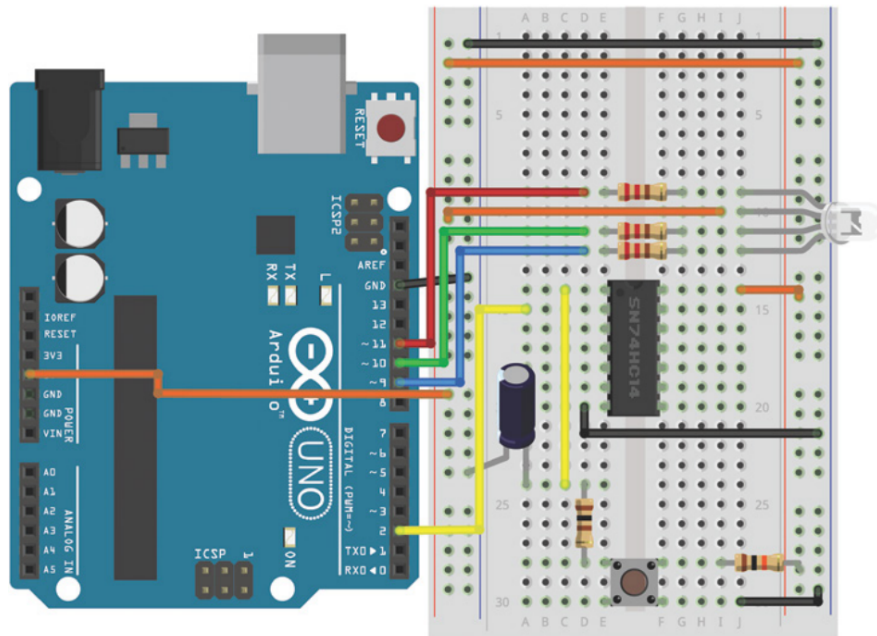
Component	Value	Purpose
Pull-up resistor R1	10 kΩ	Holds signal HIGH by default
Series resistor R2	10 kΩ	Limits capacitor discharge rate
Capacitor C	100 nF	Energy reservoir — smooths bounce transitions
Schmitt trigger	74AHCT14	Converts slow RC curve to clean digital edge; inverts signal

Hardware Interrupt Example — RGB LED Color Switcher

Concept

An RGB LED continuously fades one color up and down using `analogWrite()` with `delay()`. When the hardware-debounced button is pressed, the active color immediately changes — even mid-fade, mid-delay. This would be **impossible with polling** since `delay()` prevents the Arduino from reading inputs.

Wiring



Complete Hardware Interrupt Program

```
const int BTN  = 2; // Debounced button output → pin 2
const int RED   = 11; // Red LED cathode → pin 11
const int GREEN = 10; // Green LED cathode → pin 10
const int BLUE  = 9;  // Blue LED cathode → pin 9

// volatile: this variable is modified inside the ISR
volatile int selectedLED = RED; // Start with Red fading

void setup() {
  pinMode(RED,  OUTPUT);
  pinMode(GREEN, OUTPUT);
  pinMode(BLUE,  OUTPUT);

  // Common anode: HIGH = OFF — initialize all LEDs off
  digitalWrite(RED,  HIGH);
  digitalWrite(GREEN, HIGH);
  digitalWrite(BLUE,  HIGH);

  // Attach interrupt: pin 2, call swap() on RISING edge
  // (Schmitt trigger inverts signal: button press → RISING edge)
  attachInterrupt(digitalPinToInterrupt(BTN), swap, RISING);
}
```

```

// ISR — called immediately when button pressed
// SHORT and FAST — no delay(), no Serial.print()
void swap() {
  digitalWrite(selectedLED, HIGH); // Turn off current LED (common anode)

  // Cycle through colors: GREEN → RED → BLUE → GREEN
  if (selectedLED == GREEN) selectedLED = RED;
  else if (selectedLED == RED) selectedLED = BLUE;
  else if (selectedLED == BLUE) selectedLED = GREEN;
}

void loop() {
  // Fade selected LED UP (brightness increases)
  // Common anode: lower PWM value = brighter
  for (int i = 255; i >= 0; i--) {
    analogWrite(selectedLED, i);
    delay(10); // 10ms × 256 steps = ~2.5 sec fade up
  }

  // Fade selected LED DOWN (brightness decreases)
  for (int i = 0; i <= 255; i++) {
    analogWrite(selectedLED, i);
    delay(10); // ~2.5 sec fade down
  }

  delay(1000); // Hold briefly at minimum before next fade
}

```

Step-by-step explanation:

Code Element	Purpose
volatile int selectedLED	Must be volatile — modified in ISR, read in main loop
attachInterrupt(digitalPinToInterrupt(2), swap, RISING)	Register ISR: pin 2, function swap, fires on rising edge
swap() — no parameters, no return	Valid ISR signature
digitalWrite(selectedLED, HIGH) in ISR	Turns off current LED before switching
for loops with delay(10) in main loop	Slow fade — button presses during this are caught by interrupt
Common anode inverted logic	analogWrite(pin, 0) = full brightness; analogWrite(pin, 255) = off

When the button is pressed mid-fade, swap() runs instantly — selectedLED changes, the old LED is turned off, and on the next analogWrite() call in the loop, the new LED starts fading from its current brightness level.

Timer Interrupts

Why Timer Interrupts?

Hardware interrupts respond to **external pin events**. Timer interrupts respond to **elapsed time** — a function is called automatically every fixed number of microseconds, regardless of what the main program is doing. This enables true **background periodic tasks**.

The ATmega328P Hardware Timers

The ATmega328P has three built-in hardware timers:

Timer	Bits	Default Arduino Use
Timer0	8-bit	millis(), delay(), PWM on pins 5 and 6
Timer1	16-bit	PWM on pins 9 and 10; Servo library
Timer2	8-bit	PWM on pins 3 and 11; tone()

Timer1 is a 16-bit timer that counts from 0 to 65,535 ($2^{16} - 1$). When it overflows, it resets to 0. The rate at which it counts depends on the **clock divider** applied to the 16 MHz system clock.

Warning: Using the TimerOne library takes control of Timer1, disabling analogWrite() on pins 9 and 10 while the library is active.

The TimerOne Library

The **TimerOne** library abstracts the complexity of directly configuring Timer1 registers, allowing a simple period-based interface.

Installation: Arduino IDE → Sketch → Include Library → Manage Libraries → search "TimerOne" → Install.

TimerOne Library Functions

Function	Purpose
Timer1.initialize(period_us)	Set timer period in microseconds (1 second = 1,000,000 μ s)
Timer1.attachInterrupt(function)	Specify ISR to call every time the period elapses
Timer1.detachInterrupt()	Stop calling the ISR

Simple Timer Interrupt Blink Program

```
#include <TimerOne.h>    // Include TimerOne library

const int LED = 13;      // Onboard LED on pin 13

void setup() {
  pinMode(LED, OUTPUT);

  // Initialize timer: trigger every 1,000,000  $\mu$ s = 1 second
  Timer1.initialize(1000000);

  // Attach ISR: call blinky() every 1 second
  Timer1.attachInterrupt(blinky);
}
```

```

void loop() {
  // Any other code here executes normally
  // LED blinks independently via timer interrupt
}

// Timer ISR — called every 1 second automatically
void blinky() {
  // Toggle LED: read current state and invert it
  digitalWrite(LED, !digitalRead(LED));
}

```

Key points:

- `Timer1.initialize(1000000)` — 1,000,000 μs = 1 second period
- `blinky()` fires every 1 second regardless of what `loop()` is doing
- Any code in `loop()` executes concurrently — pseudo-multitasking
- ISR must complete **faster** than the timer period, or the main loop starves

Timer Interrupt Application — Interrupt-Driven Sound Machine

Concept

Combines **hardware interrupt** (button → changes musical key) and **timer interrupt** (automatic octave stepping every 0.5 seconds). The `loop()` independently prints the current state to Serial — all three run concurrently.

Task	Mechanism	Trigger
Step through octaves	Timer interrupt (Timer1)	Every 500 ms automatically
Change musical key	Hardware interrupt (pin 2)	Button press (rising edge)
Print current state	Main <code>loop()</code>	Continuously with <code>delay(100)</code>

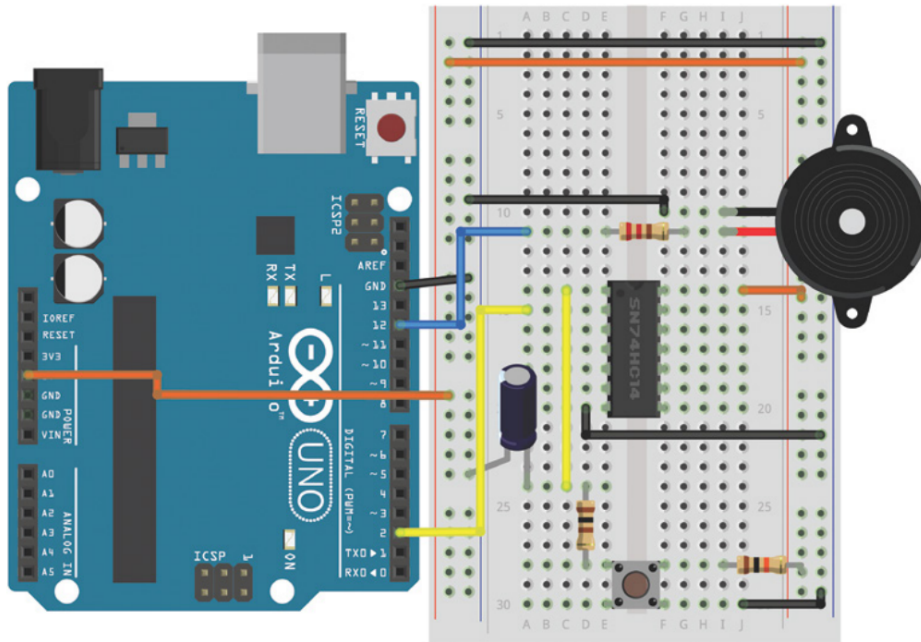
Octave Mathematics

Each octave **doubles** the frequency of the previous octave:

Octave	C Frequency	Multiplier
2	65 Hz	$\times 1$
3	130 Hz	$\times 2$
4	260 Hz	$\times 4$
5	520 Hz	$\times 8$
6	1040 Hz	$\times 16$

`octave_multiplier` starts at 1 and doubles each timer interrupt: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow$ resets to 1.

Sound Machine Wiring



Complete Sound Machine Program

```
#include <TimerOne.h> // Timer interrupt library

const int BTN    = 2; // Debounced button → pin 2
const int SPEAKER = 12; // Piezo buzzer → pin 12

// Base frequencies for octave 2 of each note
#define NOTE_C 65 // Hz
#define NOTE_D 73
#define NOTE_E 82
#define NOTE_F 87
#define NOTE_G 98
#define NOTE_A 110
#define NOTE_B 123

// volatile: both variables modified in ISRs
volatile int key          = NOTE_C; // Current musical key (base freq)
volatile int octave_multiplier = 1; // Current octave (1,2,4,8,16)

void setup() {
  Serial.begin(9600);
  pinMode(SPEAKER, OUTPUT);

  // Hardware interrupt: button press → changeKey() on RISING edge
  attachInterrupt(digitalPinToInterrupt(BTN), changeKey, RISING);

  // Timer interrupt: call changePitch() every 500,000 μs = 0.5 seconds
  Timer1.initialize(500000);
  Timer1.attachInterrupt(changePitch);
}
```



```

// Hardware ISR — called when button pressed
// Advances to next musical key; resets octave to 1
void changeKey() {
  octave_multiplier = 1;    // Reset to base octave on key change

  if (key == NOTE_C) key = NOTE_D;
  else if (key == NOTE_D) key = NOTE_E;
  else if (key == NOTE_E) key = NOTE_F;
  else if (key == NOTE_F) key = NOTE_G;
  else if (key == NOTE_G) key = NOTE_A;
  else if (key == NOTE_A) key = NOTE_B;
  else if (key == NOTE_B) key = NOTE_C; // Cycle back to C
}

// Timer ISR — called every 0.5 seconds automatically
// Steps through octaves by doubling the multiplier
void changePitch() {
  octave_multiplier = octave_multiplier * 2;    // Double frequency (next octave)
  if (octave_multiplier > 16) octave_multiplier = 1; // Reset after octave 6
  tone(SPEAKER, key * octave_multiplier);      // Play current note at current octave
}

void loop() {
  // Runs independently — prints state every 100ms
  Serial.print("Key: ");
  Serial.print(key);
  Serial.print(" Multiplier: ");
  Serial.print(octave_multiplier);
  Serial.print(" Frequency: ");
  Serial.println(key * octave_multiplier);
  delay(100);
}

```

Program behavior breakdown:

Action	What Happens
Power on	Plays C2 (65 Hz); loop prints state every 100ms
Every 0.5 seconds	Timer ISR fires: octave doubles (65→130→260→520→1040→65)
Button pressed	Hardware ISR fires: key advances (C→D→E→F→G→A→B→C), octave resets to 1
Both simultaneously	Both ISRs fire at their own independent rates — main loop unaffected

Comparing Polling, Hardware Interrupts, and Timer Interrupts

Feature	Polling	Hardware Interrupt	Timer Interrupt
Trigger	Checked in loop()	External pin state change	Elapsed time
Response time	Depends on loop speed	Immediate	At fixed interval
Code location	Inside loop()	ISR function + attachInterrupt()	ISR function + Timer1.initialize()
delay() allowed	Yes	No	No
Serial.print() allowed	Yes	No	No (technically possible but discouraged)
Missed events	Yes, if loop is slow	Never	Not applicable
Use case	Simple, fast loops	Button presses, encoders	Periodic background tasks
Library needed	None	None (built-in)	TimerOne (third-party)

MODEL QUESTIONS

1. Mention one application of timer interrupt. (3 Marks)
2. What is the use of tone() function? What are its parameters? (3 Marks)
3. Compare the benefits and drawbacks of interrupts and polling. (4 Marks)
4. Write code that will sound a buzzer when an analog sensor reading exceeds a threshold. (5 Marks)
5. Write an example application that uses the interrupt capability of Arduino. Comment your code for readability. (4 Marks)
6. Write code for playing a tone whose frequency is set by a potentiometer. (5 Marks)